

Grep

Description	Keystroke	Function	Note												
Grep in Emacs - Standard <code>grep/find</code> - Navigate findings <code>projectile grep</code> - edit finding: <code>wgrep</code> <code>fzf</code> : <code>fuzzy search</code> <code>ripgrep</code> <code>deadgrep</code>. <code>deadgrep-edit</code> <code>ag</code> See also: ↗ Dired ↗ Projectile ↗ Search/Replace			Emacs supports executing find and grep from several Emacs commands. Doing so has several advantages: - The command output is collected in the “grep” Emacs buffer while the command runs asynchronously. - The buffer is read-only, you can search within it right away. - If you type RET on a found line, Emacs visit the file of the match at the appropriate line. - Without even going inside the search result buffer you can type one of the ‘goto match’ commands to move to the next or previous match: (next-error) : C-x `, M-g n or M-g M-n (previous_error): M-g p or M-g M-p - Stop the grep asynchronous operation with C-c C-k or <f11> g k - The search commands keep a history for the searches. You can browse the history at the prompt. - Several of these commands can be executed on a remote system via ↗ Tramp (as indicated in the margin of the command for the ones I tested).  PEL supports installation and use of several grep-enhancing external packages, listed below, as controlled by PEL user-options in the pel-pkg-for-grep group. <table><tr><td> pel-use-ag</td><td> ag.el package. Requires the ag command line utility . Provides fast Ag-driven recursive searches using PCRE syntax.</td></tr><tr><td> pel-use-deadgrep ★★</td><td> deadgrep package  PEL uses my fork: it can search in multiple-directories when deadgrep-extra-searched-directories is set-up. - Requires the ripgrep  command line utility; a fast recursive file search tool. - Support searching into binary files when ripgrep-all  command line utility is available and identified as the deadgrep-executable </td></tr><tr><td> pel-use-fzf</td><td> fzf.el package. Requires the fzf command line utility . Provides fuzzy searches in files, buffers, etc... Can use ripgrep.</td></tr><tr><td> pel-use-projectile</td><td> projectile package, which uses ag and ripgrep. See ↗ Projectile.</td></tr><tr><td> pel-use-ripgrep</td><td> rg.el package.  ripgrep.el package is also installed when pel-use-projectile is on; projectile uses that. - Both require the ripgrep  command line utility; a fast recursive file search tool. </td></tr><tr><td> pel-use-wgrep</td><td> wgrep package. Provides editable search results. PEL activates in grep-mode, rg-mode and ag-mode buffers.</td></tr></table>	 pel-use-ag	 ag.el package. Requires the ag command line utility  . Provides fast Ag-driven recursive searches using PCRE syntax.	 pel-use-deadgrep ★★	 deadgrep package  PEL uses my fork: it can search in multiple-directories when deadgrep-extra-searched-directories is set-up. - Requires the ripgrep  command line utility; a fast recursive file search tool. - Support searching into binary files when ripgrep-all  command line utility is available and identified as the deadgrep-executable	 pel-use-fzf	 fzf.el package. Requires the fzf command line utility  . Provides fuzzy searches in files, buffers, etc... Can use ripgrep .	 pel-use-projectile	 projectile package, which uses ag and ripgrep . See ↗ Projectile .	 pel-use-ripgrep	 rg.el package.  ripgrep.el package is also installed when pel-use-projectile is on; projectile uses that. Both require the ripgrep  command line utility; a fast recursive file search tool.	 pel-use-wgrep	 wgrep package. Provides editable search results. PEL activates in grep-mode , rg-mode and ag-mode buffers.
 pel-use-ag	 ag.el package. Requires the ag command line utility  . Provides fast Ag-driven recursive searches using PCRE syntax.														
 pel-use-deadgrep ★★	 deadgrep package  PEL uses my fork: it can search in multiple-directories when deadgrep-extra-searched-directories is set-up. - Requires the ripgrep  command line utility; a fast recursive file search tool. - Support searching into binary files when ripgrep-all  command line utility is available and identified as the deadgrep-executable														
 pel-use-fzf	 fzf.el package. Requires the fzf command line utility  . Provides fuzzy searches in files, buffers, etc... Can use ripgrep .														
 pel-use-projectile	 projectile package, which uses ag and ripgrep . See ↗ Projectile .														
 pel-use-ripgrep	 rg.el package.  ripgrep.el package is also installed when pel-use-projectile is on; projectile uses that. Both require the ripgrep  command line utility; a fast recursive file search tool.														
 pel-use-wgrep	 wgrep package. Provides editable search results. PEL activates in grep-mode , rg-mode and ag-mode buffers.														
See ↗ Customize															
Last updated on:	2026-09-23	 ★★ PEL uses a fork of deadgrep that supports searching in multiple directory trees : search in your repo and in library trees!													
Open this PDF file. See also: ↗ Help/Info	<f11> g <f1>	(pel-help-pdf &optional OPEN-WEB-PAGE)	Open the ↗ Grep local PDF. With C-u , M-- , M- -1 or C- -1 prefix: open the remote GitHub hosted PDF instead. If the pel-flip-help-pdf-arg user-option is set it's the other way around.												
Customize PEL Grep Support See also: ↗ Customize	<f11> g <f2>	(pel-customize-pel &optional OTHER-WINDOW)	Customize PEL grep support. Use it to activate/de-activate various grep tools. If OTHER-WINDOW is non-nil (use C-u) , display in other window.												
Customize Emacs Grep Support See also: ↗ Customize	<f11> g <f3>	(pel-customize-library &optional OTHER-WINDOW)	Customize Emacs grep support. Groups: grep, ag, deadgrep, fzf, rg, ripgrep, wgrep.												
Grep Search in Emacs	Use one of the following commands to perform grep operation on files. All of these commands collect output in the “grep” buffer, - Run asynchronously, you can use any of the grep finding navigation commands while it is still running. See the list below. - To kill the grep job before it finishes, type C-c C-k. - This commands use a special history list for its arguments, so you can easily repeat a grep and find command.														
Run grep via find Works with: ↗ Tramp See also: ↗ File mngt	<f11> f g <f11> g f	(grep-find COMMAND-ARGS)	Grep in files found by the find utility: run grep via find with user-specified args COMMAND-ARGS. - Collect output in the “grep” buffer. Command has argument history; help you easily repeat a find command. - While find runs asynchronously, you can use the C-x ` to find the text that grep hits refer to.												
Run grep	<f11> g g	(grep COMMAND-ARGS)	Run grep with user-specified COMMAND-ARGS, collect output in a buffer. - Without prefix, prompts to complete a grep command. The default grep command is: grep -nH --null -e - For example to search for main in all .c files complete it like this: grep -nH --null -e main *.c - With C-u prefix creates grep search for the string at point in the same types of files as the current file.												
Local grep Works with: ↗ Tramp	<f11> g l	(lgrep REGEXP &optional FILES DIR CONFIRM)	Run grep, searching for REGEXP in FILES shell pattern in directory DIR. - This command shares argument histories with <f11> g r and <f11> g g. - FILES may use abbreviations defined in ‘grep-files-aliases’, e.g. entering ‘ch’ is equivalent to “.[ch]”. As whitespace triggers completion when entering a pattern, including it requires quoting, e.g. ‘C-q<space>’. - With C-u prefix, you can edit the constructed shell command line before it is executed. With C-u C-u prefix, directly edit and run ‘grep-command’.												
Recursive grep over files in directory tree using grep Works with: ↗ Tramp	<f11> g r	(rgrep REGEXP &optional FILES DIR CONFIRM)	Recursively grep for REGEXP in FILES in directory tree rooted at DIR. - The search is limited to file names matching shell pattern FILES. - FILES may use abbreviations defined in ‘grep-files-aliases’, e.g. entering ‘ch’ is equivalent to “.[ch]”. As whitespace triggers completion when entering a pattern, including it requires quoting, e.g. ‘C-q<space>’. - With C-u prefix, you can edit the constructed shell command line before it is executed. With C-u C-u prefix, directly edit and run ‘grep-find-command’. - This command shares argument histories with M-x lgrep and M-x grep-find. - When called programmatically and FILES is nil, REGEXP is expected to specify a command to run. With C-u prefix allows modifying the command output: e.g. for adding a sort -V so that output list the files in sorted order with lines number also sorted.												
Recursive Gzip grep Works with: ↗ Tramp	<f11> g R	(zrgrep REGEXP &optional FILES DIR CONFIRM TEMPLATE)	Recursively grep for REGEXP in gzipped FILES in tree rooted at DIR. Like ‘rgrep’ but uses ‘zgrep’ for ‘grep-program’, sets the default file name to “*.gz”, and sets ‘grep-highlight-matches’ to ‘always’. It can search into compressed files as well as regular files.												
Kill grep process	C-c C-k <f11> g k	(kill-grep)	Kill the grep process that currently runs asynchronously. Useful if you want to interrupt a long-running grep command.												
Grep findings navigation commands	Once a grep operation has been launched, the following commands can be used to navigate through the result, moving to the location of a match, inside the file at the matched line number. Fo async grep operations the commands can be used while the grep is still processing. The commands can be issued while point is inside the “grep” buffer but also outside.														
Move to first match	<f11> g l	(first-error &optional N)	Restart at the first grep found. Visit corresponding source code. With prefix arg N, visit the source code of the Nth error.												
Move to next match	C-`</b></li><li><b>C-x ` M-g n M-g M-n	(next-error &optional ARG RESET)	Move to next match. - With numeric argument ARG: specifies how many error messages to move; negative means move back to previous error messages. - With C-u prefix: reparse the error message buffer and start at the first error.												
Move to previous match	M-g p M-g M-p	(previous-error &optional N)	Move to previous match. Prefix arg N says how many error messages to move backwards (or forwards, if negative).												
Search in project with projectile See ↗ Projectile	If projectile is available, you can also search in project-related files with Projectile. With PEL, activate the projectile mode with <f11> <f8> <f8> Note that both ripgrep and ag also provide project-based searching but projectile provides more control over the concept of project.  These commands require projectile  PEL activates projectile when the pel-use-projectile user option is non-nil.														
Search in project files with recursive grep	<f8> s g	(projectile-grep &optional REGEXP ARG)	Perform rgrep in the project. - With a prefix ARG asks for files (globbing-aware) which to grep in. - With prefix ARG of ‘-’ (such as ‘M--’), default the files (without prompt), to ‘projectile-grep-default-files’. - With REGEXP given, don’t query the user for a regexp.												
Search in project files with ripgrep Rust regex syntax	<f8> s r	(projectile-ripgrep SEARCH-TERM &optional ARG)	Run a Ripgrep search with ‘SEARCH-TERM’ at current project root. With an optional prefix argument ARG SEARCH-TERM is interpreted as a regular expression.  Requires ripgrep.el package and ripgrep  CLI.  pel-use-projectile and pel-use-ripgrep .												
Search in project files with ag PCRE regex syntax	<f8> s s	(projectile-ag SEARCH-TERM &optional ARG)	Run an ag search with SEARCH-TERM in the project. With an optional prefix argument ARG SEARCH-TERM is interpreted as a regular expression.  Requires ag.el package and the ag command line utility  .  pel-use-projectile and pel-use-ag .												

Description	Keystroke	Function	Note
wgrep: edit grep buffer & write changes to files	The grep, lgrep, rgrep and zgrep search result are show in a *grep* buffer that is read-only and can be used to jump to the location of the found text. 📦 With the wgrep external package, you can activate a mode that allows you to edit the buffer and save the modification back to the files. 🔧 PEL activates wgrep when the pel-use-wgrep user option is non-nil. It also activates it when pel-use-ripgrep and pel-use-ivy are non-nil. - PEL activates support for wgrep in the grep-mode, rg-mode and ag-mode buffers. - You can perform a grep search that places result in grep-mode buffer with the following commands: grep (<f11> g g), grep-find (<f11> g f), lgrep (<f11> g l), rgrep (<f11> g r), zgrep (<f11> g z). - The RipGreg based commands: rg (<f11> g i), rg-menu (<f11> g m) - The ag based commands: ag-regexp (<f11> g a x), ag-files (<f11> g a f), ag-project-files (<f11> g a p f) and similar, ag-dired (<f11> g a d d) and similar.		
Activate wgrep in grep-mode buffer	C-c C-p C-x C-q	(wgrep-change-to-wgrep-mode)	Change to wgrep mode, allowing modifying the text to save back into file with a specific command (see below). PEL activates the C-x C-q key binding is available for grep-mode, ag-mode and rg-mode buffers. ⚠️ When the *grep* buffer is huge, this might freeze your Emacs for several minutes.
Save all changes	C-c C-s	(wgrep-save-all-buffers)	Save the buffers that wgrep changed.
Save changes to files and exit wgrep mode	C-c C-e C-c C-c C-x C-s	(wgrep-finish-edit)	Apply changes to file buffers. These changes are not immediately saved to disk unless 'wgrep-auto-save-buffer' is non-nil.
Delete line	C-c C-d	(wgrep-mark-deletion)	Mark current line as deleted. This change will be applied when C-x C-s is typed.
Discard changes & quit	C-c C-k	(wgrep-abort-changes)	Discard all changes and return to original mode.
Quit	C-x C-q	(wgrep-exit)	Return to original mode.
Toggle read-only area	C-c C-p	(wgrep-toggle-readonly-area)	Toggle read-only area to remove a whole line. See the following example: you obviously don't want to edit the first line. If grep matches a lot of lines, it's hard to edit the grep buffer. After toggling to editable, you can call 'delete-matching-lines', 'delete-non-matching-lines'. Example: <pre>----- ./svn/text-base/some.el.svn-base:87:(hoge) ./some.el:87:(hoge) -----</pre>
Remove changes in region	C-c C-r	(wgrep-remove-change BEG END)	Remove changes in the region between BEG and END.
Remove all changes	C-c C-u C-c Esc	(wgrep-remove-all-change)	Remove changes in the whole buffer.
Grep with Fuzzy File Finders See 🔍 File-mngt. , fzf manual , fzf search syntax .	The fzf command line utility is a very fast fuzzy file finder that can be used within Emacs via the fzf.el emacs front-end. It can be used with grep. To use it inside Emacs, you must: - install and configure the fzf command line utility, and 📦 Use the fzf.el external package 🔧 activated by pel-use-fzf. The fzf commands below are available when this is active. - By default all fzf grep operations use grep. It can also use the much faster ripgrep command line tool. Customize fzf/grep-command with <f11> g <f3> 4 and select ripgrep for fzf/grep-command to take advantage of that.		
Grep search files with fzf for specified regex	<f11> g z	(fzf-grep)	Prompt for string to search and file grep selection expression, show grep results in a fzf session, select appropriate line to open the specific file at appropriate line.
Grep files with specific pattern in current directory tree, then show results wit fzf. Contrl the search pattern inside a non-fuzzy fzf session.	<f11> g M-z	(fzf-grep-with-narrowing)	Call 'fzf-grep' with grep as the narrowing filter: the grep command identified by the fzf/grep-command user option is first used to find matches on the identified pattern, then FZF is used to search into the results. - By default the grep command searches in the files identified by the fzf/grep-file-pattern'user-option unless a WITH-FILE_PATTERN prefix argument argument is used; in that case it prompts for a file pattern to use. The prompt identifies the tool used (grep or rg) if it recognizes the on specified in fzf/grep-command. 👉 Remember to use the file pattern appropriate for the tool; grep and Ripgrep use different ones.
Grep search files with fzf for specified regex in specified directory	<f11> g Z	(fzf-grep-in-dir)	Prompt for directory, then for string to search and file grep selection expression, show grep results in a fzf session, select appropriate line to open the specific file at appropriate line.
Grep files with specific pattern and specific directory tree, then show results wit fzf. Contrl the search pattern inside a non-fuzzy fzf session.	<f11> g M-Z	(fzf-grep-in-dir-with-narrowing)	Call 'fzf-grep-in-dir' with grep as the narrowing filter: the grep command identified by the fzf/grep-command user option is first used to find matches on the identified pattern, then FZF is used to search into the results. - This prompts for a directory to perform the search from. - By default the grep command searches in the files identified by the fzf/grep-file-pattern'user-option unless a WITH-FILE_PATTERN prefix argument argument is used; in that case it prompts for a file pattern to use. The prompt identifies the tool used (grep or rg) if it recognizes the on specified in fzf/grep-command. 👉 Remember to use the file pattern appropriate for the tool; grep and Ripgrep use different ones.
Grep search Git repo member files with fzf for specified regex	<f11> g G	(fzf-git-grep)	Prompt for string to search inside files committed in Git repository. Show grep results over current Git repo searched in a fzf session, use fzf extra filtering text to select select appropriate line to open the specific file at appropriate line. ⚠️ This does *not* search into previous revisions of the Git committed files, it only searches the files currently in the file system that are committed.
Grep search HG repo member files and revisions with fzf for specified regex	<f11> g H	(fzf-hg-grep &optional ALL-REVS)	Grep specified versions of files committed in Mercurial repo, fzf result. Prompts for string to search, using hg grep regular expressions (Python/Perl regex). - Without prefix argument: grep files committed in Mercurial repo and perform a fzf search on the output. 👉 With C-u prefix it greps inside all revisions of the files. Perform fzf search on the grep result and open selection at specific line.
Fuzzy grep search for symbol at point	<f11> g .	(fzf-grep-dwim &optional WITH-FILE-PATTERN)	Call 'fzf-grep' on 'symbol-at-point'. - If there's no symbol at point (as identified by 'thing-at-point'), prompt for one. - By default the grep command searches in the files identified by the fzf/grep-file-pattern user-option unless a prefix argument (like C-u) is used; in that case it prompts for a file pattern to use.
Narrowed fzf-driven grep search for symbol at point	<f11> g M-.	(fzf-grep-dwim-with-narrowing &optional WITH-FILE-PATTERN)	Call 'fzf-grep' on 'symbol-at-point', with grep as the narrowing filter: the grep command identified by the fzf/grep-command user option is first used to find matches on the identified pattern, then FZF is used to search into the results. - If there's no symbol at point (as identified by 'thing-at-point'), prompt for one. - By default the grep command searches in the files identified by the 'fzf/grep-file-pattern' user-option unless WITH-FILE_PATTERN prefix argument is used; in that case it prompts for a file pattern to use.

Description	Keystroke	Function	Note
Speeding up Emacs grep searching	By default Emacs uses the standard find, grep, rgrep and zgrep command line utilities to perform the grep-based search operations. - The actual utility used depends on what is available on your Operating System.  As fast as grep might be, tools like ripgrep and ag are much faster. See the section below on ripgrep and ag. - Also note that grep-find-command user option can be modified to use ripgrep for the normal grep-find command. See notes in the reference table below.		
ripgrep front end tools Rust (ripgrep) regex syntax	PEL supports rg.el and deadgrep . They are both very useful. Each have their strong points. rg.el gives more control over the ripgrep command line options: the rg-menu command allows you to select —multiline search, for example. deadgrep outputs in buffers that can be refreshed as the content of files change. The search criteria can also be modified in the buffer. - Ability to refresh the buffer is very useful when changing names of software elements: use 2 deadgrep searches: one for the old name and one for the new name and see the first buffer deplete and the second increase as you modify your code.		
Searching with RipGrep explicitly using rg.el Rust (ripgrep) regex syntax	The following commands use ripgrep explicitly to perform various types of searches. These search are much faster than grep on large file set . - Using ripgrep through the rg.el package inside Emacs makes it very flexible with lots of features available on the command line that can be easily activated via rg-menu - To get the list of all rg command lines you can use man (or woman) inside Emacs. With PEL you can type: <f11> ? m rg RET - See ℹ Help/Info for info on man and woman. The following commands use ripgrep explicitly to perform fast grep-type searches. Using ripgrep is highly recommended.  Remember that ripgrep uses the Rust regexop syntax , not Emacs regexp syntax.  The commands require the rg.el package.  PEL activates the use of ripgrep with rg.el inside Emacs when pel-use-ripgrep user option is set to t. You must install ripgrep command line utility separately. When pel-use-projectile is set to t, PEL also installs ripgrep.el that projectile uses.		
Recursive regexp grep over files in directory tree using RipGrep Rust (ripgrep) regex syntax	<f11> g i C-c s r	(rg QUERY FILES DIR)	Run ripgrep, searching for REGEXP in FILES in directory DIR. The search is limited to file names matching shell pattern FILES.
	- FILES may use abbreviations defined in ‘rg-custom-type-aliases’ or ripgrep builtin type aliases, e.g. entering ‘elisp’ is equivalent to ‘*.el’. REGEXP is a regexp as defined by the ripgrep executable. - With C-u prefix (CONFIRM), you can edit the constructed shell command line before it is executed. - Useful to add the -z option to search into compressed files (.zip, .el.gz, etc...). - Collect output in a buffer. While ripgrep runs asynchronously, you can use C-x ` (M-x ‘next-error’), or RET in the rg output buffer, to go to the lines where rg found matches.   With PEL, only <f11> g i forces loading of the rg package. At first the C-s s r key binding is not set. It will be as soon as rg is loaded (forced by the other bindings).		
Recursive literal grep over files in directory tree using RipGrep Rust (ripgrep) regex syntax	<f11> g t C-c s t	(rg-literal QUERY FILES DIR)	Run ripgrep, searching for literal PATTERN in FILES in directory DIR. - With C-u prefix (CONFIRM), you can edit the constructed shell command line before it is executed. - Useful to add the -z option to search into compressed files (.zip, .el.gz, etc...).   With PEL, only <f11> g I forces loading of the rg package. At first the C-s s t key binding is not set. It will be as soon as rg is loaded (forced by the other bindings).
Open RipGrep transient menu	<f11> g m C-c s	(rg-menu) - While the menu is active all keys and mouse actions are controlled by the rg menu, even though point remains where it was. - The menu list options and commands shown below. - Type the options first and include the dash in what you type. Then select the command. - The current directory affects what is proposed in the directory and file type prompts of several commands. - The results are show inside a *rg* buffer. There are <u>several commands available inside the *rg* buffer</u> which provide other functionality. Type ? in the *rg* buffer for help.  The default binding C-c s is also used for several cross-reference tools such as CScope which hijacks the binding when enabled if the cscope-keymap-prefix user option is not changed. - With PEL, the PEL binding is always available in any case.	Shows the transient menu for rg.
	Search d Dwim r Regex t Literal p Project c Current directory f Current file	Manage l List s Save S Save as name	Switches -h Search hidden files (--hidden) -z Search zipped files (--search-zip) -v Invert match (--invert-match) -U Multi line (--multiline-dotall --multiline) -w Search words (--word-regexp) -l Don't cross file system (--one-file-system) -n Override ignore files (--no-ignore) Options -C Show context (--context=) -M Omit long lines (--max-columns=) -m Max matches per file (--max-count=) -g Filter files glob (--glob=) -T Exclude files types (--type-not=)
Searching with ripgrep using deadgrep ★★ - multiple-directories searches →	 The deadgrep external package uses the ripgrep command line tool (or ripgrep-all) to provide a buffer-central control of ripgrep searches.  PEL installs and activates it when the pel-use-deadgrep user-option is turned on (set to t). Use <f11> g <f2> to open the customization buffer for it. - Use <f11> g <f3> 3 to open the deadgrep customization buffer.  Set the deadgrep-extra-searched-directories to identify extra directories to search when searching inside some directory trees. - For example, use this to setup searching into libraries when searching into the directory of a project.  To search inside PDF, Word, ZIP, tar.gz and other types of files, use ripgrep-all (rga) as search command in deadgrep-executable user-option.		
Start a deadgrep search Rust (ripgrep) regex syntax	<f11> g d	(deadgrep SEARCH-TERM)	Start a ripgrep search for SEARCH-TERM. Prompt for the search string. - If called with a prefix argument, create the results buffer but don’t actually start the search. - Opens a “deadgrep” search result and control buffer where you can see the findings, navigate through them and perform other ripgrep searches with the ability to modify various search parameters. - See the deadgrep-mode commands available in this buffer in the following rows below.
Show help for deadgrep minor mode	?	(describe-mode &optional BUFFER)	Display documentation about deadgrep, with all key bindings specific to this minor mode. - Note that deadgrep evolves and the version you have may have less or more commands. - To update deadgrep with PEL, just delete (or move) the package from the ~/.emacs.d/elpa directory and restart Emacs. If pel-use-deadgrep is set, PEL will download and install the new version.
Cycle search case	C	(deadgrep-cycle-search-case)	Cycle the search case type (smart / sensitive / ignore), the restart the search.
Cycle search expression type	T	(deadgrep-cycle-search-type)	Cycle the search type (string, words, regexp), then restart the search.
Cycle search file selection type	F	(deadgrep-cycle-search-file)	Cycle the search file method (all, type, glob), then restart the search.  This is code I submitted as a PR to the Github project. It’s available in my fork of deadgrep .
Change searched expression text	S	(deadgrep-search-term)	Change the current search term and restart the search.
Change Searched Directory	D	(deadgrep-directory)	Prompt the user for a new search directory, then restart the search
Search in parent directory	^	(deadgrep-parent-directory)	Restart the search in the parent directory
Visit result at point	RET	(deadgrep-visit-result)	Goto the search result at point.
Visit result at point, in other window	o	(deadgrep-visit-result-other-window)	Goto the search result at point, opening in another window.
Re-run the search	g	(deadgrep-restart)	Re-run ripgrep with the current search settings.
Perform Incremental search on results	I	(deadgrep-incremental)	Perform an incremental search inside the deadgrep buffer.

Description	Keystroke	Function	Note
Move point to Next result/button	n	(deadgrep-forward)	Move forward to the next item. This will either be a button, a filename, or a search result. See also ‘deadgrep-forward-match’.
Move point to Previous result/button	p	(deadgrep-backward)	Move backward to the previous item. This will either be a button, a filename, or a search result. See also ‘deadgrep-backward-match’.
Move point to Next search hit	N	(deadgrep-forward-match)	Move point forward to the beginning of next match. Note that a result line may contain more than one match, or zero matches (if the result line has been truncated).
Move point to Previous search hit	P	(deadgrep-backward-match)	Move point backward to the beginning of previous match.
Move point to next file header	M-n	(deadgrep-forward-filename)	Move forward to the next filename.
Move point to previous file header	M-p	(deadgrep-backward-filename)	Move backward to the previous filename.
Collapse/Expand result for a file	<tab>	(deadgrep-toggle-file-results)	Show/hide the results of the file at point. Point must be located on a file header line.
Stop a running search	C-c C-k	(deadgrep-kill-process)	Kill the deadgrep process associated with the current buffer.
Close deadgrep window	q	(quit-window)	Close the deadgrep window and bury its buffer.
Enter deadgrep-edit-mode	C-c C-p	(deadgrep-edit-mode)	Activate deadgrep-edit-mode: allow changing content of found text in all their buffers. To store the result back to their buffers turn off the edit mode by executing deadgrep-mode (bound to C-x C-s)
Deadgrep Edit Mode	The deadgrep-edit-mode allows changing the content of strings in their respective buffers by editing the content of the deadgrep buffer.		
Save all changes and exit deadgrep-edit-mode	C-x C-s	(deadgrep)	Exit the deadgrep-edit-mode, save all changes to their respective buffers and return to the deadgrep-mode.

Searching with Ag explicitly Ag uses PCRE See also: PCRE regex syntax PCRE cheatsheet Learn PCRE in Y	The following commands use the ag command explicitly to perform various types of searches. - Some of the commands restrict the search to files of specified types. They search in field with extensions corresponding to the specified type. - For a list of the file types , execute the command ag --list-file-types. - It is also possible to provide more command lines to the ag command executed by a command. - For that, type a command prefix (like C-u) before typing the command key sequence. - For instance, you can use the -z command line switch to <i>search into zip files</i>. - To get the list of all ag command lines you can use man (or woman) inside Emacs. With PEL you can type: <f11> ? m ag RET - See ⌘ Help/Info for info on man and woman.  Requires the ag.el package and the ag command line utility.  PEL activates the use of ag inside Emacs via ag.el when pel-use-ag user option is set to t. - You must install ag command line utility separately.		
Literal search in files of specified directory tree	<f11> g a a	(ag STRING DIRECTORY)	Search using ag in a given DIRECTORY for a given literal search STRING, with STRING defaulting to the symbol under point. If called with a prefix, prompts for flags to pass to ag.
Regexp search in files of specified directory tree	<f11> g a x	(ag-regexp STRING DIRECTORY)	Search using ag in a given directory for a given regexp. - The regexp should be in PCRE syntax, not Emacs regexp syntax. - If called with a prefix, prompts for flags to pass to ag.
Literal search in files of specific types in specified directory tree	<f11> g a f	(ag-files STRING FILE-TYPE DIRECTORY)	Search using ag in a given DIRECTORY for a given literal search STRING, limited to files that match FILE-TYPE. Prompt for file type. Support tab completion. Type tab to get the list of file types. - STRING defaults to the symbol under point. - If called with a prefix, prompts for flags to pass to ag.
Ag search in project files	The following commands perform an ag search in the field of the current project directory tree. The project directory tree is identified by the presence of a (D)VCS depot file (.git, .hg, etc..) in a directory.		
Literal search in project files of specific type	<f11> g a p f	(ag-project-files STRING FILE-TYPE)	Search using ag for a given literal search STRING, limited to files that match FILE-TYPE. - Prompt for file type. Support tab completion. Type tab to get the list of file types. - STRING defaults to the symbol under point. - If called with a prefix, prompts for flags to pass to ag.
Literal search in all project files	<f11> g a p p	(ag-project STRING)	Guess the root of the current project and search it with ag for the given literal search STRING. If called with a prefix, prompts for flags to pass to ag.
Regexp Grep in all project files	<f11> g a p x	(ag-project-regexp REGEXP)	Guess the root of the current project and search it with ag for the given regexp. - The regexp should be in PCRE syntax, not Emacs regexp syntax. - If called with a prefix, prompts for flags to pass to ag.
Find files with matching name in directory tree	<f11> g a d d	(ag-dired DIR STRING)	Recursively find files in DIR matching literal search STRING. - The PATTERN is matched against the full path to the file, not only against the file name. - The results are presented as a ‘dired-mode’ buffer with ‘default-directory’ being DIR.
Find files with name matching regex in directory tree	<f11> g a d x	(ag-dired-regexp DIR REGEXP)	Recursively find files in DIR matching REGEXP. - The regexp should be in PCRE syntax, not Emacs regexp syntax. - The REGEXP is matched against the full path to the file, not only against the file name. - Results are presented as a ‘dired-mode’ buffer with ‘default-directory’ being DIR.
Find project files with a name matching regex	<f11> g a d f	(ag-project-dired-regexp REGEXP)	Recursively find files in current project matching REGEXP.
Kill all ag buffers	<f11> g a k a	(ag-kill-buffers)	Kill all ‘ag-mode’ buffers.
Kill other ag buffers	<f11> g a k o	(ag-kill-other-buffers)	Kill all ‘ag-mode’ buffers other than the current buffer.
Kill ag process	<f11> g a k p	(ag/kill-process)	Kill the ‘ag’ process running in the current buffer.

Grep under Emacs - Reference

<u>Grep</u>	The standard grep command is available on all POSIX systems. - Emacs default uses grep for the grep, zgrep and zrgrep Emacs commands. - For rgrep and zrgrep, Emacs uses a command line that invokes grep (or zgrep) via find. - The actual used command line is specified by the grep-find-template user option.
GNU EMacs Manual — Searching with Grep under Emacs	Describes Emacs basic grep support - with the use of standard GNU grep
GNU Grep 3.5 manual	GNU grep is available by default on Linux distributions. The actual version depends on the distribution.
freeBSD Grep	freeBSD grep is available on freeBSD and macOS
OpenBSD Grep	

ripgrep : rg	<u>Rust (ripgrep) regex syntax</u>	You can use ripgrep instead of GNU BSD grep to significantly speed grep searches. It's possible to use ripgrep by itself, with its own command. It is also possible to replace grep in the standard Emacs commands using grep.
Ripgreg utility: rg @ Github		Github home for the ripgrep command line utility: rg
Emacs rg.el package @ Github		Github home for rg.el
rg.el manual		Latest version of the rg.el manual (previous versions available on the same site)
ripgrep.el @ Github		Github home of ripgrep.el
<u>Blazing-fast jump-to-grep in Emacs using ripgrep</u>		Describes how the grep-find-command user option can be modified to use ripgrep for the normal grep-find command. Note that you do not need to write Emacs code as described in that article, all you have to do is customize gre-find-command user option and change the value. By default, grep-find-command is nil, but is set by Emacs to: <pre>'("find . -type f -exec grep -nH --null -e \\{\\} +" . 42)</pre> Also note that changing grep-find-command will not impact the behaviour of the zgrep command, so it will still be slow. A better solution this problem is to leave grep-find-command alone and use the rg.el package which provides fast ripgrep searches, including gripping into zip files.
<u>Ripgrep-all : rga</u>		Ripgrep extension: uses ripgrep to search inside several other file types: PDF, Word, ZIP, tar.gz, etc...
• Deadgrep	<u>Rust (ripgrep) regex syntax</u>	deadgrep is another, very useful, ripgrep front end for Emacs. Search result show in a refreshable buffer with option control. 🙌 it can also use Ripgrep-all (rga), which means that you can search inside PDF files and several others!
deadgrep @ Github		Home page
Ag	<u>PCRE regex syntax</u>	The ag.el package is a front end for the ag, “ <i>The Silver Searcher</i> ” command line utility, which uses PCRE.
Ag @ GitHub		Github home of the ag command line utility.
ag.el @ Github		Github home for the Emacs package that provides access to the ag command line utility.
Online regexp testers		There are several online regexp tools. Some are shown below. • <u>Debuggex</u> : Javascript, Python, PCRE. With <u>cheatsheets</u>. With visualization of regexp graph. Match rendering. • <u>regex101</u> : PCRE, Javascript, Python, Golang, Java 8. With cheatsheet. Textual explanation of regexp. Match rendering. • <u>regexr</u> : Javascript, PCRE. Text graph rendering. • <u>RexV.2</u> : PHP PCRE, PHP Posix, Perl, Python, Javascript, Node.JS. Cheatsheet. Flags control.